Project Documentation: AdventureWorks Analytics Pipeline

Enterprise Analytics Layer & Executive Dashboard

Problem Statement

The AdventureWorks database is optimized for daily operational transactions, not for business intelligence. Executives and analysts need fast, reusable reporting datasets, but repeatedly querying the raw OLTP schema is slow, brittle, and hard to maintain.

Goal

Design and implement a reusable analytics layer on top of the enterprise database, build a chained SQL pipeline from raw tables to executive KPI views, and expose only dashboard-ready analytical datasets to the notebook.

Solution Approach

The solution uses a layered analytics architecture inside PostgreSQL. Raw operational tables are transformed into dimensions, facts, aggregations, and then executive KPI datasets. The notebook reads only the final analytical views, ensuring the raw database is not queried repeatedly.

Analytics Architecture

• Layer 1: Core Dimensions - Product, Customer, Territory, Employee, Vendor, Date

• Layer 2: Core Facts - Sales, Purchases, Inventory

• Layer 3: Analytical Aggregations - Customer metrics, Product metrics, Employee performance, Regional analysis

• Layer 4: Executive KPI Datasets - Sales, Customers, Products, Employees, Territories, Inventory

Business Domains Used

• Sales

• Production

• Purchasing

• HumanResources

• Person

• Territory

• Vendor

Reusable Analytical Views

The project creates more than 10 reusable views within the analytics schema. Each view is built on

prior intermediate outputs, avoiding repeated calculations and enforcing a dependency chain.

• analytics.dim_product

• analytics.dim_customer

• analytics.dim_territory

• analytics.dim_employee

• analytics.dim_vendor

- analytics.dim_date
- analytics.fact_sales
- analytics.fact_inventory
- analytics.fact_purchases
- analytics.customer_metrics
- analytics.product_metrics
- analytics.employee_performance
- analytics.regional_analysis
- analytics.kpi_sales
- analytics.kpi_customers
- analytics.kpi_products
- analytics.kpi_employees
- analytics.kpi_territories
- analytics.kpi_inventory

Advanced SQL Techniques

- Chained Common Table Expressions (CTEs)
- Window Functions including LAG and ranking functions
- CASE WHEN expressions for business categorization
- Conditional Aggregation for segmented revenue and retention metrics
- Complex JOINs across multiple business domains
- Reusable intermediate views to prevent duplicated calculation

Notebook Dashboard

The Jupyter notebook connects to PostgreSQL, reads only the final analytical views using pandas.read_sql(), and generates executive visualizations. It contains at least eight charts with business insights.

- Revenue Trend
- Sales by Territory
- Customer Segments
- Top Product Performance
- Category Revenue
- Employee Performance Leaderboard
- Inventory Health Status
- Executive KPI Summary

Business Insights

- Revenue is concentrated in specific months, indicating seasonality and the need for demand management.
- A small number of territories drive most revenue, showing geographic concentration.
- Customer value is skewed toward medium and low-value segments, creating upsell opportunities.
- Product revenue is driven by a subset of top-selling SKUs, which is both a strength and a risk.
- Sales performance is dominated by top reps, suggesting a need to replicate best practices across the team.
- Inventory analysis shows low-stock and overstocked products, enabling better purchasing decisions.

Executive Recommendations

• Focus investment and marketing on top-performing territories.

• Promote best-selling products and bundle related SKUs.

• Target Medium/Low value customers with loyalty and upsell campaigns.

• Scale top sales behaviors through coaching and incentives.

• Balance inventory by replenishing low-stock items and reducing overstock.

Challenges Faced

• Highly normalized operational schema requiring many joins.

• Missing derived columns after loading data into PostgreSQL, requiring manual metric reconstruction.

• Designing a chained pipeline that reuses intermediate outputs without repetition.

• Aligning SQL output with executive storytelling in notebook visualizations.

Assumptions Made

• All monetary values are treated as USD.

• Date dimension covers 2000-2030 to support all transactional dates.

• The source dataset contains complete header-detail relationships with no orphaned row